communicate between threads, the future objects themselves don't provide synchronized accesses. If multiple threads need to access a single future object, they must protect access via a mutex or other synchronization mechanism, as described in chapter 3. However, as you'll see in section 4.2.5, multiple threads may each access their own copy of a `std::shared_future<>` without further synchronization, even if they all refer to the same asynchronous result.

The most basic of one-off events is the result of a calculation that has been run in the background. Back in chapter 2 you saw that `std::thread` doesn't provide an easy means of returning a value from such a task, and I promised that this would be addressed in chapter 4 with futures—now it's time to see how.

4.2.1. Returning values from background tasks

Suppose you have a long-running calculation that you expect will eventually yield a useful result but for which you don't currently need the value. Maybe you've found a way to determine the answer to Life, the Universe, and Everything, to pinch an example from Douglas Adams.[1] You could start a new thread to perform the calculation, but that means you have to take care of transferring the result back, because `std::thread` doesn't provide a direct mechanism for doing so. This is where the `std::async` function template (also declared in the `<future>` header) comes in.

[1] In *The Hitchhiker's Guide to the Galaxy*, the computer Deep Thought is built to determine "the answer to Life, the Universe and Everything." The answer is 42.

You use `std::async` to start an *asynchronous task* for which you don't need the result right away. Rather than giving you back a `std::thread` object to wait on, `std::async` returns a `std::future` object, which will eventually hold the return value of the function. When you need the value, you just call `get()` on the future, and the thread blocks until the future is *ready* and then returns the value. The following listing shows a simple example.

Listing 4.6. Using `std::future` to get the return value of an asynchronous task

```
#include <future>
#include <iostream>

int find_the_answer_to_ltuae();
void do_other_stuff();
int main()
{
    std::future<int> the_answer=std::async(find_the_answer_to_ltuae);
    do_other_stuff();
    std::cout<<"The answer is "<<the_answer.get()<<std::endl;
}
```

`std::async` allows you to pass additional arguments to the function by adding extra arguments to the call, in the same way that `std::thread` does. If the first argument is a pointer to a member function, the second argument provides the object on which to apply the member function (either directly, or via a pointer, or wrapped in `std::ref`), and the remaining arguments are passed as arguments to the member function. Otherwise, the second and subsequent arguments are passed as arguments to the function or callable object specified as the first argument. Just as with `std::thread`, if the arguments are rvalues, the copies are created by *moving* the originals. This